CRYPTOGRAPHY LAB

Complete Lab Exam Answers - All Sets

Short Java programs using built-in functions wherever possible

SET-1: ADVANCED ENCRYPTION STANDARD (AES)

Q1. Explain the Advanced Encryption Standard (AES) algorithm, including SubBytes, ShiftRows, MixColumns and AddRoundKey.

AES (Advanced Encryption Standard) is a symmetric-key block cipher used to protect digital data. It operates on 128-bit blocks and supports 128-bit, 192-bit and 256-bit secret keys.

- AES-128: 128-bit key, 10 rounds
- AES-192: 192-bit key, 12 rounds
- AES-256: 256-bit key, 14 rounds

The 128-bit input block is arranged as a 4 x 4 byte state matrix. AES encryption uses the following transformations:

1. AddRoundKey - The current state is XORed with a round key derived from the original key. It is performed before the first round and in every round.

2. SubBytes - Each byte is replaced using a fixed substitution table called the S-Box. This provides confusion.

3. ShiftRows - Rows of the state matrix are cyclically shifted left: row 0 by 0 bytes, row 1 by 1 byte, row 2 by 2 bytes and row 3 by 3 bytes.

4. MixColumns - Each column is transformed using matrix multiplication in the finite field GF(2^8). This spreads the effect of each byte and provides diffusion.

5. AddRoundKey - The transformed state is XORed with the corresponding round key.

In the final round, MixColumns is omitted. AES decryption uses the inverse operations: InvShiftRows, InvSubBytes, AddRoundKey and InvMixColumns.

Q2. Java program for AES encryption and decryption

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.*;

public class AESDemo {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter message: ");
        String msg = sc.nextLine();

        SecretKeySpec key = new SecretKeySpec("1234567890123456".getBytes(), "AES");
        Cipher c = Cipher.getInstance("AES/ECB/PKCS5Padding");

        c.init(Cipher.ENCRYPT_MODE, key);
        String enc = Base64.getEncoder().encodeToString(c.doFinal(msg.getBytes()));

        c.init(Cipher.DECRYPT_MODE, key);
        String dec = new String(c.doFinal(Base64.getDecoder().decode(enc)));
```

```
        System.out.println("Encrypted: " + enc);
        System.out.println("Decrypted: " + dec);
    }
}
```

Sample Output

```
Enter message: Hello
Encrypted: <Base64 encrypted value>
Decrypted: Hello
```

Important Viva Points

Why is AES symmetric? The same secret key is used for encryption and decryption.

AES block size? 128 bits.

AES key sizes? 128, 192 and 256 bits.

SET-2: IMAGE STEGANOGRAPHY

Q1. Explain RC4 Algorithm.

RC4 (Rivest Cipher 4) is a symmetric stream cipher. It generates a pseudorandom byte stream and XORs that keystream with plaintext to produce ciphertext. RC4 has two main stages:

1. Key Scheduling Algorithm (KSA): An array S containing 0 to 255 is initialized and shuffled using the secret key.

$$j = (j + S[i] + Key[i \bmod KeyLength]) \bmod 256$$

2. Pseudo Random Generation Algorithm (PRGA): The shuffled S array is repeatedly updated to generate keystream bytes.

$$Ciphertext = Plaintext \text{ XOR } Keystream$$

$$Plaintext = Ciphertext \text{ XOR } Keystream$$

RC4 is simple and fast, but it has known security weaknesses and should not be used for modern secure systems.

Q2. Demonstrate QuickStego to hide and retrieve a text message from an image.

Aim

To hide a secret text message inside an image using QuickStego and retrieve the hidden message.

Procedure to Hide Text

1. Open QuickStego.
2. Click Open Image and select an image file.
3. Type or load the secret text message.
4. Click Hide Text.
5. Click Save Image and save the steganographic image with a new name.

Procedure to Retrieve Text

6. Open QuickStego.
7. Open the image containing the hidden text.
8. Click Get Text.
9. The hidden text is displayed.

Result

The secret text message was successfully hidden inside the image and retrieved using QuickStego.

Important Viva Points

What is steganography? It is the technique of hiding secret information inside another medium such as an image, audio or video.

Cryptography vs steganography? Cryptography hides the meaning of data, while steganography hides the existence of data.

SET-3: SHA-512 SECURE HASH ALGORITHM

Q1. Explain the working of SHA-512 secure hash algorithm and its major processing steps.

SHA-512 is a cryptographic hash function in the SHA-2 family. It converts input of any length into a fixed 512-bit (64-byte) digest, normally written as 128 hexadecimal characters.

1. Padding - A 1 bit is appended followed by enough 0 bits so that the length becomes 896 mod 1024.

2. Append Length - The original message length is appended as a 128-bit value, making the final length a multiple of 1024 bits.

3. Initialize Hash Values - Eight 64-bit initial hash values H0 to H7 are used.

4. Divide into Blocks - The message is processed in 1024-bit blocks. Each block starts with sixteen 64-bit words.

5. Message Schedule - The sixteen words are expanded to eighty 64-bit words W0 to W79.

6. Compression - Each block is processed through 80 rounds using XOR, AND, NOT, right rotations, shifts, Ch and Maj functions.

7. Final Hash - The final eight 64-bit hash values are concatenated to produce a 512-bit digest.

$$Ch(x,y,z) = (x \text{ AND } y) \text{ XOR } ((\text{NOT } x) \text{ AND } z)$$

$$Maj(x,y,z) = (x \text{ AND } y) \text{ XOR } (x \text{ AND } z) \text{ XOR } (y \text{ AND } z)$$

Q2. Java program to generate SHA-512 hash

```
import java.security.*;
import java.math.BigInteger;
import java.util.*;

public class SHA512Demo {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter message: ");
        byte[] hash = MessageDigest.getInstance("SHA-512").digest(sc.nextLine().getBytes());
        System.out.printf("%0128x%n", new BigInteger(1, hash));
    }
}
```

Sample Output

```
Enter message: Hello
3615f80c9d293ed7402687f94b22d58e...
```

Important Viva Points

SHA-512 output size? 512 bits.

Can SHA-512 be decrypted? No. Hashing is a one-way operation.

SHA-512 block size? 1024 bits.

Number of rounds? 80 rounds.

SET-4: ELLIPTIC CURVE POINT OPERATIONS

Q1. Explain the basic principles of elliptic curve cryptography.

Elliptic Curve Cryptography (ECC) is a public-key cryptographic technique based on arithmetic with points on elliptic curves over finite fields.

$$y^2 = x^3 + ax + b \pmod p$$

The curve must satisfy the non-singularity condition 4a^3 + 27b^2 != 0.

Point Addition: For two points P and Q, their sum R = P + Q is another point on the curve.

Point Doubling: When P = Q, the tangent at P is used to calculate 2P.

Point Subtraction: P - Q = P + (-Q), where -Q = (x, -y mod p).

Scalar Multiplication: Repeated addition gives Q = kP and is fundamental to ECC.

$$\text{For } P \neq Q: \lambda = (y_2 - y_1) / (x_2 - x_1) \bmod p$$

$$x_3 = \lambda^2 - x_1 - x_2 \bmod p$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \bmod p$$

$$\text{For } P = Q: \lambda = (3x_1^2 + a) / (2y_1) \bmod p$$

ECC security is based on the Elliptic Curve Discrete Logarithm Problem (ECDLP). It offers strong security with smaller key sizes than many older public-key systems.

Q2. Java program for elliptic curve point addition and subtraction

Example curve: y^2 = x^3 + 2x + 2 (mod 17)

```
import java.math.BigInteger;

public class ECCDemo {
    static BigInteger p=BigInteger.valueOf(17), a=BigInteger.valueOf(2);

    static class P {
        BigInteger x,y;
        P(long x,long y){this(BigInteger.valueOf(x),BigInteger.valueOf(y));}
        P(BigInteger x,BigInteger y){this.x=x;this.y=y;}
        public String toString(){return "("+x+","+y+")";}
    }

    static P add(P A,P B){
        if(A==null)return B; if(B==null)return A;
        if(A.x.equals(B.x)&&A.y.add(B.y).mod(p).equals(BigInteger.ZERO))return null;
        BigInteger m=A.equals(B)?BigInteger.ZERO:
            (A.x.equals(B.x)&&A.y.equals(B.y)?
            A.x.pow(2).multiply(BigInteger.valueOf(3)).add(a)
            .multiply(A.y.multiply(BigInteger.TWO).mod(p).modInverse(p)).mod(p):
            B.y.subtract(A.y).multiply(B.x.subtract(A.x).mod(p).modInverse(p)).mod(p));
        BigInteger x=m.pow(2).subtract(A.x).subtract(B.x).mod(p);
        return new P(x,m.multiply(A.x.subtract(x)).subtract(A.y).mod(p));
    }

    static P neg(P q){return new P(q.x,p.subtract(q.y).mod(p));}

    public static void main(String[] args){
        P P1=new P(5,1), P2=new P(6,3);
        System.out.println("P + Q = "+add(P1,P2));
        System.out.println("P - Q = "+add(P1,neg(P2)));
```

```
    }
}
```

Note: For an exam, if you prefer maximum reliability, write the same formulas shown above along with this program logic. The output for the chosen points is:

```
P + Q = (10,6)
P - Q = (5,16)
```

Important Viva Points

General ECC equation? y^2 = x^3 + ax + b mod p.

Point subtraction? P - Q = P + (-Q).

Main hard problem in ECC? Elliptic Curve Discrete Logarithm Problem.

SET-5: RSA ALGORITHM

Q1. Explain the RSA public-key cryptographic algorithm.

RSA is an asymmetric cryptographic algorithm developed by Rivest, Shamir and Adleman. It uses a public key for encryption and a private key for decryption.

10. Choose two large prime numbers p and q.
11. Compute n = p x q.
12. Compute phi(n) = (p - 1)(q - 1).
13. Choose e such that 1 < e < phi(n) and gcd(e, phi(n)) = 1.
14. Compute d = e^-1 mod phi(n).

$$\text{Public Key} = (e, n)$$

$$\text{Private Key} = (d, n)$$

$$\text{Encryption: } C = M^e \bmod n$$

$$\text{Decryption: } M = C^d \bmod n$$

RSA is used for secure communication, digital signatures, authentication and key transport.

Q2. Java program for RSA encryption and decryption

```
import java.security.*;
import javax.crypto.Cipher;
import java.util.*;

public class RSADemo {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter message: ");
        String msg = sc.nextLine();

        KeyPairGenerator g = KeyPairGenerator.getInstance("RSA");
        g.initialize(2048);
        KeyPair kp = g.generateKeyPair();

        Cipher c = Cipher.getInstance("RSA/ECB/PKCS1Padding");
        c.init(Cipher.ENCRYPT_MODE, kp.getPublic());
        byte[] enc = c.doFinal(msg.getBytes());

        c.init(Cipher.DECRYPT_MODE, kp.getPrivate());
        byte[] dec = c.doFinal(enc);

        System.out.println("Encrypted: " + Base64.getEncoder().encodeToString(enc));
        System.out.println("Decrypted: " + new String(dec));
    }
}
```

Sample Output

```
Enter message: Hello
Encrypted: <Base64 encrypted value>
Decrypted: Hello
```

Important Viva Points

RSA full form? Rivest, Shamir and Adleman.

RSA type? Asymmetric cryptography.

Which key encrypts? Public key.

Which key decrypts? Private key.

Common modern RSA key size? 2048 bits or higher.

SET-6: DIFFIE-HELLMAN KEY EXCHANGE

Q1. Explain the Diffie-Hellman key exchange mechanism with a suitable example.

Diffie-Hellman allows two parties to establish the same shared secret over an insecure channel without directly transmitting that secret.

15. Alice and Bob publicly agree on a prime p and a generator g.
16. Alice chooses private value a; Bob chooses private value b.
17. Alice computes A = g^a mod p; Bob computes B = g^b mod p.
18. They exchange A and B.
19. Alice computes K = B^a mod p; Bob computes K = A^b mod p. Both obtain the same shared secret.

Example

$$p = 23, g = 5, a = 6, b = 15$$

$$A = 5^6 \bmod 23 = 8$$

$$B = 5^15 \bmod 23 = 19$$

$$\text{Alice: } 19^6 \bmod 23 = 2$$

$$\text{Bob: } 8^15 \bmod 23 = 2$$

Therefore the shared secret is 2. Basic Diffie-Hellman does not authenticate the communicating parties, so it can be vulnerable to a man-in-the-middle attack if used without authentication.

Q2. Java program for Diffie-Hellman key exchange

```
import java.security.*;
import java.util.*;
import javax.crypto.KeyAgreement;
import javax.crypto.interfaces.DHPublicKey;
import javax.crypto.spec.DHParameterSpec;

public class DHDemo {
    public static void main(String[] args) throws Exception {
        KeyPairGenerator g = KeyPairGenerator.getInstance("DH");
        g.initialize(512);
        KeyPair alice = g.generateKeyPair();

        DHParameterSpec p = ((DHPublicKey)alice.getPublic()).getParams();
        g.initialize(p);
        KeyPair bob = g.generateKeyPair();

        KeyAgreement a = KeyAgreement.getInstance("DH");
        a.init(alice.getPrivate());
        a.doPhase(bob.getPublic(), true);

        KeyAgreement b = KeyAgreement.getInstance("DH");
        b.init(bob.getPrivate());
        b.doPhase(alice.getPublic(), true);

        byte[] s1 = a.generateSecret(), s2 = b.generateSecret();
        System.out.println("Alice Secret: " + Base64.getEncoder().encodeToString(s1));
        System.out.println("Bob Secret  : " + Base64.getEncoder().encodeToString(s2));
        System.out.println("Same Secret : " + Arrays.equals(s1, s2));
    }
}
```

Sample Output

```
Alice Secret: <generated secret>
Bob Secret  : <same generated secret>
Same Secret : true
```

Important Viva Points

What is Diffie-Hellman used for? Secure key exchange.

Does it directly encrypt messages? No. It establishes a shared secret.

What values are public? p, g and the public values A and B.

Main weakness without authentication? Man-in-the-middle attack.

QUICK REVISION TABLE

Set / Topic	Main Java Built-in	Key Fact
AES	Cipher, SecretKeySpec	128-bit block; 128/192/256-bit keys
Image Steganography / RC4	QuickStego practical	RC4 uses KSA + PRGA
SHA-512	MessageDigest	512-bit output; 80 rounds
ECC	BigInteger.modInverse()	P - Q = P + (-Q)
RSA	KeyPairGenerator, Cipher	Public encrypts; private decrypts
Diffie-Hellman	KeyAgreement, KeyPairGenerator	Both generate the same shared secret

Important Values to Remember

AES

- Block size: 128 bits
- Key sizes: 128 / 192 / 256 bits
- Rounds: 10 / 12 / 14

SHA-512

- Output: 512 bits
- Block size: 1024 bits
- Rounds: 80
- Digest: 128 hexadecimal characters

RSA

- Public key: (e,n)
- Private key: (d,n)
- Encryption: C = M^e mod n
- Decryption: M = C^d mod n

ECC

- Equation: y^2 = x^3 + ax + b mod p
- Subtraction: P - Q = P + (-Q)

Diffie-Hellman

- A = g^a mod p
- B = g^b mod p
- Shared secret: B^a mod p = A^b mod p